

COURSE CODE/TITLE:	CPE 212 - Automating Server Management	SECTION:	CPE31S2
DATE PERFORMED:	09 - 03 - 26	DATE SUBMITTED:	09 - 04 - 26
NAME:	Santiago, David Owen A.	INSTRUCTOR:	Engr. Jimlord Quejado

Hands-On Activity 6.1	Targeting Specific Nodes
1. PROCEDURE OUTPUT	
Task 1: Targeting Specific Nodes 1. Create a new playbook and name it site.yml. Follow the commands as shown in the image below. Make sure to save the file and exit. <pre> --- - hosts: all become: true tasks: - name: install apache and php for Ubuntu servers apt: name: - apache2 - libapache2-mod-php state: latest update_cache: yes when: ansible_distribution == "Ubuntu" - name: install apache and php for CentOS servers dnf: name: - httpd - php state: latest when: ansible_distribution == "CentOS" </pre> OUTPUT	

```

24
25 - name: Install Apache2 package and PHP Support Ubuntu
26   tags: apache,apache2,ubuntu
27   apt:
28     name:
29       - apache2
30       - libapache2-mod-php
31     state: latest
32     update_cache: yes
33   when: ansible_distribution == "Ubuntu" # for Ubuntu
34
35 - name: Install Apache2 package and PHP Support CentOS
36   tags: apache,centos,httpd
37   dnf:
38     name:
39       - httpd
40       - php
41     state: latest
42     update_cache: yes
43   when: ansible_distribution == "CentOS" # for CentOS
44

```

2. Edit the inventory file. Remove the variables we put in our last activity and group according to the image shown below:

```

[web_servers]
192.168.56.120
192.168.56.121

[db_servers]
192.168.56.122

[file_servers]
192.168.56.123

```

Make sure to save the file and exit.

OUTPUT

```
24
25     web_servers:
26     |   hosts:
27     |   |   ubuntu1:
28     |   |   |   ansible_host: 192.168.56.102
29     |   |   |   ansible_user: server1
30
31     db_servers:
32     |   hosts:
33     |   |   ubuntu2:
34     |   |   |   ansible_host: 192.168.56.103
35     |   |   |   ansible_user: server2
36
37     file_servers:
38     |   hosts:
39     |   |   centos1:
40     |   |   |   ansible_host: 192.168.56.104
41     |   |   |   ansible_user: santiago
42
```

Right now, we have created groups in our inventory file and put each server in its own group. In other cases, you can have a server be a member of multiple groups, for example you have a test server that is also a web server.

3. Edit the *site.yml* by following the image below

```

---
- hosts: all
  become: true
  pre_tasks:
    - name: install updates (CentOS)
      dnf:
        update_only: yes
        update_cache: yes
        when: ansible_distribution == "CentOS"
    - name: install updates (Ubuntu)
      apt:
        upgrade: dist
        update_cache: yes
        when: ansible_distribution == "Ubuntu"

- hosts: web_servers
  become: true
  tasks:
    - name: install apache and php for Ubuntu servers
      apt:
        name:
          - apache2
          - libapache2-mod-php
        state: latest
        when: ansible_distribution == "Ubuntu"
    - name: install apache and php for CentOS servers
      dnf:
        name:
          - httpd
          - php
        state: latest
        when: ansible_distribution == "CentOS"

```

Make sure to save the file and exit.

The *pre-tasks* command tells the ansible to run it before any other thing. In the *pre-tasks*, CentOS will install updates while Ubuntu will upgrade its distribution package. This will run before running the second play, which is targeted at *web_servers*. In the second play, apache and php will be installed on both Ubuntu servers and CentOS servers.

Run the *site.yml* file and describe the result.

OUTPUT

```
ansible-lab > ! site.yml
1  ---
2  - hosts: all
3    become: true
4    pre_tasks:
5
6    - name: install updates (CentOS)
7      tags: always
8      dnf:
9        update_only: yes
10       update_cache: yes
11       when: ansible_distribution == "CentOS"
12
13    - name: install updates (Ubuntu)
14      tags: always
15      apt:
16        upgrade: dist
17        update_cache: yes
18        when: ansible_distribution == "Ubuntu"
19
20
21  - hosts: web_servers
22    become: true
23    tasks:
24
25    - name: Install Apache2 package and PHP Support Ubuntu
26      tags: apache,apache2,ubuntu
27      apt:
28        name:
29          - apache2
30          - libapache2-mod-php
31        state: latest
32        update_cache: yes
33        when: ansible_distribution == "Ubuntu" # for Ubuntu
34
35    - name: Install Apache2 package and PHP Support CentOS
36      tags: apache,centos,httpd
37      dnf:
38        name:
39          - httpd
40          - php
41        state: latest
42        update_cache: yes
43        when: ansible_distribution == "CentOS" # for CentOS
44
45
```

```
TASK [install updates (Ubuntu)] *****
skipping: [centos1]
changed: [ubuntu2]
changed: [ubuntu1]

PLAY [web_servers] *****

TASK [Gathering Facts] *****
ok: [ubuntu1]

TASK [Install Apache2 package and PHP Support Ubuntu] *****
changed: [ubuntu1]

TASK [Install Apache2 package and PHP Support CentOS] *****
skipping: [ubuntu1]

PLAY RECAP *****
centos1      : ok=2    changed=0    unreachable=0    failed=0    skipped=1    r
escued=0    ignored=0
ubuntu1     : ok=4    changed=2    unreachable=0    failed=0    skipped=2    r
escued=0    ignored=0
ubuntu2     : ok=2    changed=1    unreachable=0    failed=0    skipped=1    r
escued=0    ignored=0
```

The pre-tasks are the first tasks to execute before executing the rest of the plays.

4. Let's try to edit the *site.yml* file again. This time, we are going to add plays targeting the other servers. This time we target the *db_servers* by adding it on the current *site.yml*. Below is an example: (Note add this at the end of the playbooks from task 1.3.

```
- hosts: db_servers
  become: true
  tasks:

    - name: install mariadb package (CentOS)
      yum:
        name: mariadb-server
        state: latest
      when: ansible_distribution == "CentOS"

    - name: "Mariadb- Restarting/Enabling"
      service:
        name: mariadb
        state: restarted
        enabled: true

    - name: install mariadb package (Ubuntu)
      apt:
        name: mariadb-server
        state: latest
      when: ansible_distribution == "Ubuntu"
```

```

46 - hosts: db_servers
47   become: true
48   tasks:
49
50   - name: install mariadb package (CentOS)
51     #tags: centos, db, mariadb
52     dnf:
53       name: mariadb-server
54       state: latest
55     when: ansible_distribution == "CentOS"
56
57   - name: install mariadb package (Ubuntu)
58     tags: db, mariadb, ubuntu
59     apt:
60       name: mariadb-server
61       state: latest
62     when: ansible_distribution == "Ubuntu"
63
64   - name: "Mariadb- Restarting/Enabling"
65     service:
66       name: mariadb
67       state: restarted
68       enabled: true

```

Make sure to save the file and exit.

Run the *site.yml* file and describe the result.

OUTPUT

```

TASK [Gathering Facts] *****
ok: [ubuntu2]

TASK [install mariadb package (CentOS)] *****
skipping: [ubuntu2]

TASK [install mariadb package (Ubuntu)] *****
changed: [ubuntu2]

TASK [Mariadb- Restarting/Enabling] *****
changed: [ubuntu2]

PLAY RECAP *****
centos1      : ok=2    changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0
ubuntu1     : ok=4    changed=1    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
ubuntu2     : ok=5    changed=3    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0

```

Because the plays only target specific groups in the inventory file, the servers in

these groups are the only ones receiving the changes from the play.

5. Go to the remote server (Ubuntu) terminal that belongs to the `db_servers` group and check the status for mariadb installation using the command: **`systemctl status mariadb`**. Do this on the CentOS server also.

Describe the output.

```
server2@server2:~$ systemctl status mariadb
● mariadb.service - MariaDB 11.8.6 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; preset: ▶
   Active: active (running) since Thu 2026-09-03 23:11:56 UTC; 1min 2s ago
   Invocation: 5c5cca597a5e421ea897562f1e6c107c
     Docs: man:mariabdb(8)
           https://mariadb.com/kb/en/library/systemd/
   Process: 7049 ExecStartPre=/bin/sh -c [ ! -e /usr/bin/galera_recovery ] && ▶
   Process: 7116 ExecStartPost=/bin/rm -f /run/mysqld/wsrep-start-position /ru▶
   Process: 7118 ExecStartPost=/etc/mysql/debian-start (code=exited, status=0/▶
  Main PID: 7102 (mariabdb)
    Status: "Taking your SQL requests now..."
     Tasks: 11 (limit: 27083)
    Memory: 93M (peak: 97.6M)
       CPU: 1.227s
    CGroup: /system.slice/mariadb.service
            └─7102 /usr/sbin/mariabdb

lines 1-16/16 (END)
```

MariaDB was successfully installed in the remote server using the playbook from the workstation.

6. Edit the **`site.yml`** again. This time we will append the code to configure installation on the **`file_servers`** group. We can add the following on our file

```
- hosts: file_servers
  become: true
  tasks:

  - name: install samba package
    package:
      name: samba
      state: latest
```

Make sure to save the file and exit.

Run the *site.yml* file and describe the result.

OUTPUT

```

72 - hosts: file_servers
73   become: true
74   tasks:
75
76   - name: install samba package
77     tags: samba
78     package:
79       name: samba
80       state: latest

```

```

PLAY [file_servers] *****

TASK [Gathering Facts] *****
ok: [centos1]

TASK [install samba package] *****
changed: [centos1]

PLAY RECAP *****
centos1 : ok=4    changed=1    unreachable=0    failed=0    skipped=1    r
escued=0  ignored=0
ubuntu1 : ok=4    changed=1    unreachable=0    failed=0    skipped=2    r
escued=0  ignored=0
ubuntu2 : ok=5    changed=2    unreachable=0    failed=0    skipped=2    r
escued=0  ignored=0

```

Samba was successfully installed on the file_servers group which contains my CentOS machine, as reflected in the recap

The testing of the *file_servers* is beyond the scope of this activity, and as well as our topics and objectives. However, in this activity we were able to show that we can target hosts or servers using grouping in ansible playbooks.

Task 2: Using Tags in running playbooks

In this task, our goal is to add metadata to our plays so that we can only run the plays that we want to run, and not all the plays in our playbook.

1. Edit the *site.yml* file. Add tags to the playbook. After the name, we can place the tags:
name_of_tag. This is an arbitrary command, which means you can use any name for a tag.

```

---
- hosts: all
  become: true
  pre_tasks:

```

```
- hosts: web_servers
  become: true
  tasks:

  - name: install apache and php for Ubuntu servers
    tags: apache,apache2,ubuntu
    apt:
      name:
        - apache2
        - libapache2-mod-php
      state: latest
    when: ansible_distribution == "Ubuntu"

  - name: install apache and php for CentOS servers
    tags: apache,centos,httpd
    dnf:
      name:
        - httpd
        - php
      state: latest
    when: ansible_distribution == "CentOS"
```

```
- hosts: db_servers
  become: true
  tasks:

  - name: install mariadb package (CentOS)
    tags: centos, db, mariadb
    dnf:
      name: mariadb-server
      state: latest
    when: ansible_distribution == "CentOS"

  - name: "Mariadb- Restarting/Enabling"
    service:
      name: mariadb
      state: restarted
      enabled: true

  - name: install mariadb package (Ubuntu)
    tags: db, mariadb, ubuntu
    apt:
      name: mariadb-server
      state: latest
    when: ansible_distribution == "Ubuntu"

- hosts: file_servers
  become: true
  tasks:

  - name: install samba package
    tags: samba
    package:
      name: samba
      state: latest
```

Make sure to save the file and exit.

Run the *site.yml* file and describe the result.

```
PLAY [file_servers] *****
TASK [Gathering Facts] *****
ok: [centos1]

TASK [install samba package] *****
ok: [centos1]

PLAY RECAP *****
centos1 : ok=4 changed=0 unreachable=0 failed=0 skipped=1 r
escued=0 ignored=0
ubuntu1 : ok=4 changed=1 unreachable=0 failed=0 skipped=2 r
escued=0 ignored=0
ubuntu2 : ok=5 changed=2 unreachable=0 failed=0 skipped=2 r
escued=0 ignored=0
```

The playbook still executed normally since we didn't specify any tags while running it. All of the plays were executed sequentially.

2. On the local machine, try to issue the following commands and describe each result:

2.1 *ansible-playbook --list-tags site.yml*

```
● santiago@workstation:~/CPE212-Santiago/ansible-lab$ ansible-playbook --list-tags site.yml

playbook: site.yml

play #1 (all): all TAGS: []
TASK TAGS: [always]

play #2 (web_servers): web_servers TAGS: []
TASK TAGS: [apache, apache2, centos, httpd, ubuntu]

play #3 (db_servers): db_servers TAGS: []
TASK TAGS: [centos, db, mariadb, ubuntu]

play #4 (file_servers): file_servers TAGS: []
TASK TAGS: [samba]
```

This play lists all of tags per play in the playbook.

2.2 *ansible-playbook --tags centos --ask-become-pass site.yml*

```
PLAY [web_servers] *****

TASK [Gathering Facts] *****
ok: [ubuntu1]

TASK [Install Apache2 package and PHP Support CentOS] *****
skipping: [ubuntu1]

PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [ubuntu2]

TASK [install mariadb package (CentOS)] *****
skipping: [ubuntu2]

PLAY [file_servers] *****

TASK [Gathering Facts] *****
ok: [centos1]

PLAY RECAP *****
centos1      : ok=3    changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0
ubuntu1      : ok=3    changed=1    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
ubuntu2      : ok=3    changed=1    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0
```

This command only executed the plays containing the centos tag. It skipped the rest.

*2.3 **ansible-playbook --tags db --ask-become-pass site.yml***

```
TASK [install updates (Ubuntu)] *****
skipping: [centos1]
changed: [ubuntu1]
changed: [ubuntu2]

PLAY [web_servers] *****

TASK [Gathering Facts] *****
ok: [ubuntu1]

PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [ubuntu2]

TASK [install mariadb package (CentOS)] *****
skipping: [ubuntu2]

TASK [install mariadb package (Ubuntu)] *****
ok: [ubuntu2]

PLAY [file_servers] *****

TASK [Gathering Facts] *****
ok: [centos1]

PLAY RECAP *****
centos1      : ok=3    changed=0    unreachable=0    failed=0    skipped=1
escued=0     ignored=0
ubuntu1      : ok=3    changed=1    unreachable=0    failed=0    skipped=1
escued=0     ignored=0
ubuntu2      : ok=4    changed=1    unreachable=0    failed=0    skipped=2
escued=0     ignored=0
```

This command only executed the plays containing the db tag. It skipped the rest.

2.4 *ansible-playbook --tags apache --ask-become-pass site.yml*

```
santiago@workstation:~/CPE212-Santiago/ansible-lab$ ansible-playbook --tags apache --ask-become-pass site.yml
skipping: [ubuntu2]
ok: [centos1]

TASK [install updates (Ubuntu)] *****
skipping: [centos1]
changed: [ubuntu1]
changed: [ubuntu2]

PLAY [web_servers] *****

TASK [Gathering Facts] *****
ok: [ubuntu1]

TASK [Install Apache2 package and PHP Support Ubuntu] *****
ok: [ubuntu1]

TASK [Install Apache2 package and PHP Support CentOS] *****
skipping: [ubuntu1]

PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [ubuntu2]

PLAY [file_servers] *****

TASK [Gathering Facts] *****
ok: [centos1]

PLAY RECAP *****
centos1 : ok=3    changed=0    unreachable=0    failed=0    skipped=1
escued=0  ignored=0
ubuntu1 : ok=4    changed=1    unreachable=0    failed=0    skipped=2
escued=0  ignored=0
ubuntu2 : ok=3    changed=1    unreachable=0    failed=0    skipped=1
escued=0  ignored=0
```

This command only executed the plays containing the apache tag. It skipped the rest.

*2.5 **ansible-playbook --tags "apache,db" --ask-become-pass site.yml***

```
santiago@workstation:~/CPE212-Santiago/ansible-lab$ ansible-playbook --tags "apache,db" --ask
-become-pass site.yml
changed: [ubuntu2]

PLAY [web_servers] *****

TASK [Gathering Facts] *****
ok: [ubuntu1]

TASK [Install Apache2 package and PHP Support Ubuntu] *****
ok: [ubuntu1]

TASK [Install Apache2 package and PHP Support CentOS] *****
skipping: [ubuntu1]

PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [ubuntu2]

TASK [install mariadb package (CentOS)] *****
skipping: [ubuntu2]

TASK [install mariadb package (Ubuntu)] *****
ok: [ubuntu2]

PLAY [file_servers] *****

TASK [Gathering Facts] *****
ok: [centos1]

PLAY RECAP *****
centos1 : ok=3 changed=0 unreachable=0 failed=0 skipped=1 r
escued=0 ignored=0
ubuntu1 : ok=4 changed=1 unreachable=0 failed=0 skipped=2 r
escued=0 ignored=0
ubuntu2 : ok=4 changed=1 unreachable=0 failed=0 skipped=2 r
escued=0 ignored=0
```

This command only executed the plays containing both the apache and db tag. It skipped the rest.

Task 3: Managing Services

1. Edit the file site.yml and add a play that will automatically start the httpd on CentOS server.

```
- name: install apache and php for CentOS servers
tags: apache,centos,httpd
dnf:
  name:
    - httpd
    - php
  state: latest
when: ansible_distribution == "CentOS"

- name: start httpd (CentOS)
tags: apache, centos,httpd
service:
  name: httpd
  state: started
when: ansible_distribution == "CentOS"
```

Figure 3.1.1

Make sure to save the file and exit.

OUTPUT

```
34
35 - name: Install Apache2 package and PHP Support CentOS
36 tags: apache, centos, httpd
37 dnf:
38   name:
39     - httpd
40     - php
41   state: latest
42   update_cache: yes
43 when: ansible_distribution == "CentOS" # for CentOS
44
45 - name: start httpd (CentOS)
46 tags: apache, centos, httpd
47 service:
48   name: httpd
49   state: started
50   enabled: true
51 when: ansible_distribution == "CentOS"
```

You would also notice from our previous activity that we already created a module that runs a service.

```
- hosts: db_servers
  become: true
  tasks:

- name: install mariadb package (CentOS)
  tags: centos, db,mariadb
  dnf:
    name: mariadb-server
    state: latest
  when: ansible_distribution == "CentOS"

- name: "Mariadb- Restarting/Enabling"
  service:
    name: mariadb
    state: restarted
    enabled: true
```

Figure 3.1.2

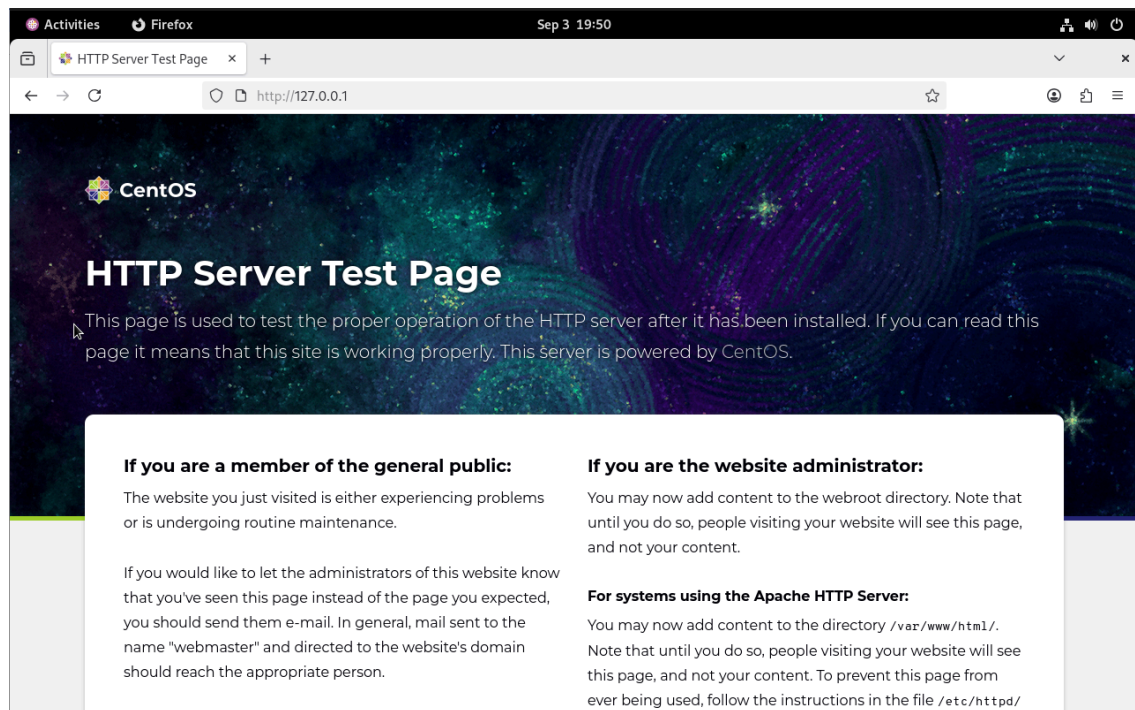
This is because in CentOS, installed packages' services are not run automatically. Thus, we need to create the module to run it automatically.

2. To test it, before you run the saved playbook, go to the CentOS server and stop the currently running httpd using the command *sudo systemctl stop httpd*. When prompted, enter the sudo password. After that, open the browser and enter the CentOS server's IP address. You should not be getting a display because we stopped the httpd service already.
3. Go to the local machine and this time, run the *site.yml* file. Then after running the file, go again to the CentOS server and enter its IP address on the browser. Describe the result.

```
santiago@workstation:~/CPE212-Santiago/ansible-lab$ ansible-playbook --tags centos sit

TASK [Gathering Facts] *****
ok: [centos1]

PLAY RECAP *****
centos1      : ok=3    changed=0    unreachable=0    failed=0    skipped
escued=0     ignored=0
ubuntu1      : ok=0    changed=0    unreachable=1    failed=0    skipped
escued=0     ignored=0
ubuntu2      : ok=0    changed=0    unreachable=1    failed=0    skipped
escued=0     ignored=0
```



As seen in the output, the ansible playbook is also capable of running a service installed on a remote system. It can be further automated by automatically adding the enable line for each package/service installation.

To automatically enable the service every time we run the playbook, use the command **enabled: true** similar to Figure 7.1.2 and save the playbook.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Supplementary Activity:

1. Creating a Role for Web Servers

```
● santiago@workstation:~/CPE212-Santiago/ansible-lab$ ansible-galaxy
init web
- Role web was created successfully
```

2. Creating a Role for Database Servers

```
● santiago@workstation:~/CPE212-Santiago/ansible-lab$ ansible-galaxy
init db
- Role db was created successfully
```

3. Combining Multiple Roles in a Playbook New site.yml

```
18 | when: ansible_distribution == Ubuntu
19 |
20 |
21 | - hosts: web_servers
22 |   become: true
23 |   roles:
24 |     - role: web
25 |     tags: web
26 |
27 | - hosts: db_servers
28 |   become: true
29 |   roles:
30 |     - role: db
31 |     tags: db
32 |
33 | - hosts: file_servers
34 |   become: true
35 |   tasks:
36 |
37 |     - name: install samba package
38 |       tags: samba
39 |       package:
40 |         name: samba
41 |         state: latest
42 |
43 |
```

Tasks per role:

Web_servers:

```
1  #SPDX-License-Identifier: MIT-0
2  ---
3  # tasks file for web
4
5  - name: Install Apache2 package and PHP Support Ubuntu
6    tags: apache, apache2, ubuntu
7    apt:
8      name:
9        - apache2
10       - libapache2-mod-php
11      state: latest
12      update_cache: yes
13      when: ansible_distribution == "Ubuntu" # for Ubuntu
14
15  - name: Install Apache2 package and PHP Support CentOS
16    tags: apache, centos, httpd
17    dnf:
18      name:
19        - httpd
20        - php
21      state: latest
22      update_cache: yes
23      when: ansible_distribution == "CentOS" # for CentOS
24
25  - name: start httpd (CentOS)
26    tags: apache, centos, httpd
27    service:
28      name: httpd
29      state: started
30      enabled: true
31    when: ansible_distribution == "CentOS"
```

Db_servers:

```
4
5 - name: install mariadb package (CentOS)
6   tags: centos, db, mariadb
7   dnf:
8     name: mariadb-server
9     state: latest
10  when: ansible_distribution == "CentOS"
11
12 - name: install mariadb package (Ubuntu)
13   tags: db, mariadb, ubuntu
14   apt:
15     name: mariadb-server
16     state: latest
17  when: ansible_distribution == "Ubuntu"
18
19 - name: "Mariadb- Restarting/Enabling"
20   service:
21     name: mariadb
22     state: restarted
23     enabled: true
```

Adding new roles automatically created their own subdirectories in the main ansible directory. Inside are other subdirectories, defining their tasks, defaults, handlers, etc. Since these roles are now separate, the tasks in the universal site.yml file also need to be moved to the specific roles' task.yml file.

To orchestrate multiple roles in a single playbook for different server groups. Add tags to each role to allow selective execution, e.g., tags: web for webserver, tags: db for database.

OUTPUT

```

santiago@workstation:~/CPE212-Santiago/ansible-lab$ ansible-playbook --tags "web,db" site.yml
skipping: [ubuntu1]

TASK [web : start httpd (CentOS)] *****
skipping: [ubuntu1]

TASK [web : Print Confirm if Successful] *****
ok: [ubuntu1] => {
  "msg": "Apache and PHP are installed and running successfully."
}

PLAY [db_servers] *****

TASK [Gathering Facts] *****
ok: [ubuntu2]

TASK [db : install mariadb package (CentOS)] *****
skipping: [ubuntu2]

TASK [db : install mariadb package (Ubuntu)] *****
ok: [ubuntu2]

TASK [db : Mariadb- Restarting/Enabling] *****
changed: [ubuntu2]

TASK [db : Print Confirm if Successful] *****
ok: [ubuntu2] => {
  "msg": "MariaDB is installed and running successfully."
}

PLAY [file_servers] *****

TASK [Gathering Facts] *****
ok: [centos1]

PLAY RECAP *****
centos1      : ok=3    changed=0    unreachable=0    failed=0    skipped=1    rescued=0    ignored=0
ubuntu1      : ok=5    changed=1    unreachable=0    failed=0    skipped=3    rescued=0    ignored=0
ubuntu2      : ok=6    changed=2    unreachable=0    failed=0    skipped=2    rescued=0    ignored=0

```

The playbook was able to execute the role-specific plays and skip the rest.

Reflections:

Answer the following:

1. What is the importance of putting our remote servers into groups?

Putting remote servers into groups creates a better management environment because each task is isolated to one group. It allows the system to continue running even if one module fails because it will not affect every single server, only the affected ones. Moreover, it enhances security especially with plays that require privilege escalation, isolating them into separate groups to prevent security breaches.

2. What is the importance of tags in playbooks?

Tags serve as another filtering option for running playbooks. It allows us to only execute specific plays without the need to directly edit the playbook. It helps in troubleshooting and making sure no other nodes or groups are affected when we need to execute only specific plays.

3. Why do you think some services need to be managed automatically in playbooks?

Some services need to be running constantly to ensure little-to-no downtime in the system. As such, ensuring services are running constantly is an important task in managing servers which is best done using playbooks. Playbooks can run perpetually to ensure a system is always in execution, meaning it should also be used to manage services that have to always be active, further maintaining automation.

GITHUB: <https://github.com/owen-s-dotdev/CPE212-Santiago.git>

4. CONCLUSION/ANALYSIS/LEARNINGS

In conclusion, this activity explored the automation and management capabilities of ansible playbooks. Here, I learned how to implement groups, tags, roles, and filters to only execute specific plays for specific groups. This helps maintain security and automation especially for systems that are running 24/7. Moreover, I observed the importance of isolating groups of nodes from each other; it helps maintain security by having their tasks separate from each other while also maintaining automation since failure in these groups will not cause failure to the entire system.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Gemini AI

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Debugging VM issues

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: AI was only used to debug the VM issues for the managed nodes. The code analysis and supplementary are all entirely my own and from the activity.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.



By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.